CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 5

Registration Form with Frontend Validation and User-Friendly Error Handling

HTML5 · CSS3 · JavaScript (ES6+) · Regular Expressions · PHP 8 server-side validation

Corrected submission – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

Corrected report. This report replaces the original Practical 5 report (`Practical-5/PRAC_5_ITUE203_25CS023.docx`, kept unchanged in the repository). The original report described the standalone, static page `Practical-5/register.html`, which sent and stored nothing. This version describes the same real-time regular-expression validation as it now runs on the portal's real registration page `studenthub/register.php`, where the PHP server validates every value again before it creates the account. The Canvas CAPTCHA of the standalone page was deliberately not carried over (Section 10). All screenshots in this report were taken from the canonical integrated application `studenthub/` with Microsoft Edge on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Aim and Objectives

To give the StudentHub student registration form accessible, real-time client-side validation in JavaScript using regular expressions, with clear error messages placed next to each field and a password strength meter – while the PHP server remains the final authority and checks every value again before an account is created.

- Validate every field of the portal's registration form – full name, username, e-mail address, gender, course, admission year, password, confirm password and terms acceptance – in JavaScript with regular expressions and rules.
- Use suitable HTML5 input types and attributes (`type="email"`, `type="password"`, `required`, `minlength`, `maxlength`, `pattern`, `autocomplete`).
- Show a specific, helpful message directly below the invalid field, worded like the server's own message.
- Check password strength with a live meter while the user types.
- Intermediate extension: validate in real time – once a field has been left, re-check it on every change.
- Block the submission while a rule is broken and move the focus to the first invalid field.
- Keep the server-side validation unchanged, so a request that bypasses the browser is still refused.
- Record tests and genuine browser screenshots of invalid and valid input.

The advanced extension of the practical list (a custom CAPTCHA drawn on an HTML5 Canvas) was implemented in the standalone `Practical-5/` folder but is intentionally not part of the portal; the reason is given in Section 10.

2. Problem Definition

Official problem definition: “Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.”

Practical 5 was first built as the standalone page `Practical-5/register.html`. It validated in the browser only, and its own success message stated that nothing was sent or saved. The integrated portal `studenthub/` already had a working registration page, `register.php` (added when Practical 9 was integrated), but according to the audit in `TEST_REPORT.md` (section 12) it had “server-side validation only (no client-side checks)”. The task of this integration was therefore to bring the Practical 5 ideas – regular-expression checks, messages next to the fields, real-time feedback and a strength meter – to the real form, without weakening the server checks that protect the database.

The practical list asks four key questions; the answers for the integrated implementation are:

Key question	Answer in the integrated portal
Are correct input types and attributes used?	Yes: `type="text"` (name; username with a `pattern`), `type="email"`, `type="password"` with `minlength="8"`, `maxlength="72"` and

	autocomplete="new-password", radio buttons for gender, selects for course and admission year and a checkbox for the terms; required on every field. The form carries novalidate, so the page's own messages are used instead of the browser's pop-ups.
Are validation errors displayed near the relevant fields?	Yes. Each message is shown in a <p class="field-error"> whose id is the field name plus -error (for example id="email-error"), directly below its field and linked to the field with aria-describedby. The server uses the same element ids, so the script reuses a server message element if one is already on the page.
Is password strength checked?	Yes. A meter with five levels (Very weak, Weak, Fair, Good, Strong) is updated on every key press. Whether the password is accepted is decided separately by the password policy: 8–72 characters with an uppercase letter, a lowercase letter, a number and a special character.
Is the form accessible using keyboard and screen-reader-friendly labels?	Every control has a visible label (for/id), the gender radio buttons are in a role="radiogroup" with aria-label="Gender", invalid fields get aria-invalid="true", the strength label is an aria-live="polite" region, and the focus moves to the first invalid field when the form is submitted.

3. Tools and Technologies

Tool / technology	Use in this practical
HTML5	studenthub/register.php: form controls, input types, validation attributes, ARIA attributes
CSS3	studenthub/assets/css/auth.css: field messages and the password strength meter (lines 120–126)
JavaScript (ES6+)	studenthub/assets/js/register-validation.js: regular expressions, event listeners, strength meter, submit blocking
PHP 8.2.12 (XAMPP)	studenthub/includes/registration.php and studenthub/includes/auth.php: server-side validation, CSRF token, password hashing, the registration transaction
MariaDB 10.4.32	Tables users and students of the portal database studenthub_db
Python 3 (standard library)	tests/test_forms.py: HTTP tests against the PHP built-in server and a disposable database
Playwright + Microsoft Edge (headless)	tests/browser_forms.py: browser checks and the screenshots in Section 8
Git / GitHub	Version control; the integration is commit 938e74f of the StudentHub repository
Operating system	Windows 11

4. Place in the Integrated StudentHub Portal

Practical 5 is no longer a separate page. Its validation is attached to the portal's public registration page, which is reached from the login page and creates a real student account. The files involved are:

File / route	Role
studenthub/register.php (route register.php)	Public registration page. A signed-in user is redirected to the dashboard (administrators to the Admin Console). The form posts to itself with a CSRF token; invalid input returns HTTP 422 with the messages under the fields.
studenthub/assets/js/register-validation.js	The Practical 5 script, loaded on register.php with defer (line 64).

studenthub/assets/css/auth.css	Styles of .field-error and of the strength meter (.strength, .strength-bar).
studenthub/includes/registration.php	validate_registration() (server rules) and register_student_account() (duplicate checks and the transaction).
studenthub/includes/auth.php	password_problems(), PASSWORD_POLICY_TEXT, csrf_field() / csrf_is_valid(), login_user() and e().
tests/test_forms.py, tests/browser_forms.py	Tests of the integration (Section 9).
evidence/practical-05-07-integration/01…03	Genuine Edge screenshots (Section 8).

How a registration flows through the portal:

1. The visitor opens register.php. While typing, register-validation.js checks each field and shows the messages; the form is not sent while a rule is broken.
2. When all browser checks pass, the form is sent with POST together with the session's CSRF token.
3. register.php checks the token (csrf_is_valid()) and calls validate_registration($_POST), which repeats every rule on the server.
4. If the values are valid, register_student_account() creates the login account in users (role always student) and the linked record in students in one transaction (Section 6).
5. The new user is signed in with login_user() (new session ID and CSRF token), a USER_REGISTER entry is written to the audit log, and the user is redirected to dashboard.php with a welcome message containing the new enrollment number.

In this way Practical 5 (client-side validation) works together with the integrated Practical 9 (secure registration and password hashing), Practical 10 (sessions and roles) and Practical 14 (audit log).

5. Implementation

5.1 HTML5 input types and validation attributes

The form (<form method="POST" action="register.php" novalidate>, lines 102–173 of register.php) is generated by PHP so that, after a server-side refusal, the entered values (never the passwords) and the server's messages are shown again. Every value is printed through e() (HTML escaping). The controls and their attributes are:

Field	Control and HTML5 attributes in register.php
Full name	type="text", required, minlength="2", maxlength="100", autocomplete="name"
Username	type="text", required, minlength="3", maxlength="30", pattern="[A-Za-z][A-Za-z0-9_]*", autocomplete="username"
E-mail address	type="email", required, maxlength="100", autocomplete="email", spellcheck="false"
Gender	three type="radio" buttons (Male, Female, Other) in a role="radiogroup"; Male is preselected
Course	select, required, empty first option "-- Select Course --"; Information Technology, Computer Science Engineering, Data Science, Web Development
Admission year	select, required; the current academic year and the two before it (registration_admission_years())
Password	type="password", required, minlength="8", maxlength="72", autocomplete="new-password", aria-describedby="password-help" (the policy text)

Confirm password	`type="password"`, `required`, `minlength="8"`, `maxlength="72"`, `autocomplete="new-password"`
Terms acceptance	`type="checkbox"`, `value="1"`, `required`

5.2 Regular expressions and the password rules

The script defines three regular expressions and a function that lists the missing password rules. The rules are the same as the server's (`includes/registration.php` and `password_problems()` in `includes/auth.php`). The name pattern uses Unicode property escapes (`\p{L}` letters, `\p{M}` combining marks), so names such as “Anne-Marie” or names with accented letters are accepted. The 72-character limit is counted in bytes with `TextEncoder`, like PHP's `strlen()`, because bcrypt uses only the first 72 bytes of a password.

studenthub/assets/js/register-validation.js (lines 9–26) – regular expressions and password rules

```
 9      const NAME = /^\p{L}[\p{L}\p{M} .'-]*$/u;
10      const USERNAME = /^[A-Za-z][A-Za-z0-9_]*$/;
11      const EMAIL = /^[^\s@<>()[\]\\,;:"]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$/;
12      const touched = new Set();
13
14      const value = (name) => (form.elements[name] && form.elements[name].value) || '';
15      const byteLength = (text) => new TextEncoder().encode(text).length;
16
17      // Missing password rules, worded like the server's password_problems()
18      function passwordProblems(pw) {
19          const problems = [];
20          if (pw.length < 8) problems.push('at least 8 characters');
21          if (byteLength(pw) > 72) problems.push('at most 72 characters');
22          if (!/[A-Z]/.test(pw)) problems.push('an uppercase letter');
23          if (!/[a-z]/.test(pw)) problems.push('a lowercase letter');
24          if (!/[0-9]/.test(pw)) problems.push('a number');
25          if (!/[^A-Za-z0-9]/.test(pw)) problems.push('a special character');
26          return problems;
```

Field	Browser rule (register-validation.js)	Server rule (registration.php / auth.php)
Full name	spaces collapsed and trimmed; 2–100 characters; `/^\p{L}[\p{L}\p{M} .'-]*$/u`	the same steps with `preg_replace`, `mb_strlen` and the same pattern in `preg_match`
Username	trimmed; 3–30 characters; `/^[A-Za-z][A-Za-z0-9_]*$/`	the same length and pattern
E-mail	trimmed; at most 100 characters; the `EMAIL` regular expression	lower-cased; at most 100 characters; `filter_var($email, FILTER_VALIDATE_EMAIL)`
Gender / course / year	a radio button checked; a course and a year selected	value must be in `REGISTRATION_GENDERS`, `REGISTRATION_COURSES` or `registration_admission_years()`
Password	`passwordProblems()`: at least 8 characters, at most 72 bytes, A–Z, a–z, 0–9 and a character other than A–Z, a–z and 0–9	`password_problems()`: the same five checks
Confirm password	equal to the password (checked only once the password itself is valid)	`hash_equals()` with the password, only if the password is valid
Terms	checkbox ticked	`agree` must be exactly `"1"`

The only rule that is not literally identical is the e-mail check: the browser uses a regular expression, the server PHP's `filter_var()`. If the two ever disagree, the server's answer counts, because every submission is checked again on the server. The messages are worded like the server's, except that the server adds examples to the special-character rule (“a special character such as ! @ # ?”).

5.3 One validator per field

`rules` maps each field name to a function that returns the error message, or `null` when the value is valid. Because the messages are the server's texts, a user sees the same wording whether the problem is found in the browser or on the server.

studenthub/assets/js/register-validation.js (lines 30–61) – the validators of all nine fields

```
30      const rules = {
31          name() {
32              const v = value('name').trim().replace(/\s+/g, ' ');
33              if (!v) return 'Please enter your full name.';
34              if (v.length < 2 || v.length > 100) return 'The name must be 2–100 characters long.';
35              return NAME.test(v) ? null : 'Use letters, spaces, apostrophes, dots and hyphens only.';
36          },
37          username() {
38              const v = value('username').trim();
39              if (!v) return 'Please choose a username.';
40              if (v.length < 3 || v.length > 30) return 'The username must be 3–30 characters long.';
41              return USERNAME.test(v) ? null : 'Start with a letter; then use letters, numbers and
    underscores only.';
42          },
43          email() {
44              const v = value('email').trim();
45              if (!v) return 'Please enter your email address.';
46              return v.length <= 100 && EMAIL.test(v) ? null : 'Please enter a valid email address,
    e.g. name@example.com.';
47          },
48          gender: () => (form.querySelector('input[name="gender"]:checked') ? null : 'Please choose
    Male, Female or Other.'),
49          course: () => (value('course') ? null : 'Please select a course from the list.'),
50          adm_year: () => (value('adm_year') ? null : 'Please select an admission year from the
    list.'),
51          password() {
52              const pw = value('password');
53              if (!pw) return 'Please choose a password.';
54              const problems = passwordProblems(pw);
55              return problems.length ? 'The password needs ' + problems.join(', ') + '.' : null;
56          },
57          confirm_password() {
58              if (rules.password()) return null;                      // first fix the password itself
59              return value('confirm_password') === value('password') ? null : 'The passwords do not
    match.';
60          },
61          agree: () => (form.elements.agree.checked ? null : 'Please accept the Terms & Conditions.'),
```

5.4 Showing the message next to the field

`errorElement()` (lines 65–75) reuses the server's message element (for example `id="name-error"`) if the page already contains one, or creates a new `<p class="field-error">` inside the field's `.form-group` (after the terms row for the checkbox). `show()` writes the message with `textContent` (never as HTML), hides the element when the field becomes valid, and sets or removes `aria-invalid="true"`; the message id is added to `aria-describedby`, so a screen reader reads the message with the field. For the gender radio buttons the attributes are set on all three buttons.

studenthub/assets/js/register-validation.js (lines 77–97) – show() and check()

```
77      function show(field, message) {
78          const input = form.elements[field];
79          const inputs = input instanceof RadioNodeList ? [...input] : [input];
80          if (!message && !document.getElementById(field + '-error')) return;
81          const el = errorElement(field);
82          el.textContent = message || '';
83          el.hidden = !message;
84          inputs.forEach((i) => {
85              if (message) {
86                  i.setAttribute('aria-invalid', 'true');
87                  if (!(i.getAttribute('aria-describedby') || '').includes(el.id))
    i.setAttribute('aria-describedby', ((i.getAttribute('aria-describedby') || '') + ' ' +
    el.id).trim());
88              } else {
89                  i.removeAttribute('aria-invalid');
```

```
            }
        });
    }

    function check(field) {
        const message = rules[field]();
        show(field, message);
        return message === null;
```

5.5 Real-time validation and submit blocking (intermediate extension)

- **On blur:** when the user leaves a field it is marked as *touched* (a `Set`) and checked.
- **On input:** text, e-mail and password fields are re-checked on every key press once they are touched, so a message disappears as soon as the mistake is corrected, and a field the user has not reached yet is not marked in red.
- **On change:** radio buttons, the checkbox and the selects are checked as soon as they change.
- **Linked fields:** typing in the password updates the strength meter and re-checks the confirm-password field if it was touched.
- **On submit:** all fields are checked and marked as touched. If any rule is broken, `event.preventDefault()` stops the request and the focus moves to the first invalid field (for gender, the first radio button).

studenthub/assets/js/register-validation.js (lines 121–144) – input, change and blur listeners and the submit handler

```
    // Real-time: after a field was left once, it is re-checked on every change
    for (const field of Object.keys(rules)) {
        const input = form.elements[field];
        const inputs = input instanceof RadioNodeList ? [...input] : [input];
        inputs.forEach((i) => {
            i.addEventListener('blur', () => { touched.add(field); check(field); });
            i.addEventListener(i.type === 'radio' || i.type === 'checkbox' || i.tagName === 'SELECT' ? 'change' : 'input', () => {
                if (field === 'password') { updateMeter(); if (touched.has('confirm_password')) check('confirm_password'); }
                if (touched.has(field) || i.type === 'radio' || i.type === 'checkbox' || i.tagName === 'SELECT') check(field);
            });
        });
    }

    form.addEventListener('submit', (event) => {
        const invalid = Object.keys(rules).filter((field) => !check(field));
        invalid.forEach((field) => touched.add(field));
        if (invalid.length) {
            event.preventDefault();
            const first = form.elements[invalid[0]];
            (first instanceof RadioNodeList ? first[0] : first).focus();
        }
    });
    updateMeter();
})();
```

5.6 Password strength meter

The meter is inserted by the script after the password policy text (`#password-help`). Its score counts five points: at least 8 characters, at least 12 characters, both upper- and lower-case letters, a digit, and a special character. Any non-empty password scores at least 1. The bar width is the score × 20 %, and the label is announced politely to screen readers.

studenthub/assets/js/register-validation.js (lines 100–118) – the strength meter

```
    // Password strength meter (independent of the pass/fail rules above)
    const meter = document.createElement('div');
    meter.className = 'strength';
    meter.innerHTML = '<span class="strength-bar"><span></span></span><span class="strength-label" aria-live="polite"></span>';
    document.getElementById('password-help').after(meter);
```

```
105      const LEVELS = [['', ''], ['Very weak', 'very-weak'], ['Weak', 'weak'], ['Fair', 'fair'],
     ['Good', 'good'], ['Strong', 'strong']];
106      function updateMeter() {
107          const pw = value('password');
108          let score = 0;
109          if (pw.length >= 8) score++;
110          if (pw.length >= 12) score++;
111          if (/[A-Z]/.test(pw) && /[a-z]/.test(pw)) score++;
112          if (/[0-9]/.test(pw)) score++;
113          if (/[^A-Za-z0-9]/.test(pw)) score++;
114          if (pw && score === 0) score = 1;
115          const [label, cls] = LEVELS[pw ? Math.max(1, score) : 0];
116          meter.dataset.level = cls;
117          meter.querySelector('.strength-bar span').style.width = (pw ? Math.max(1, score) * 20 : 0) +
     '%';
118          meter.querySelector('.strength-label').textContent = pw ? 'Password strength: ' + label : '';
```

Score	Label shown	Bar (auth.css)	Example verified by the Edge test
1	Very weak	red (`--danger`), 20 %	`abc` (Figure 1)
2	Weak	red (`--danger`), 40 %	–
3	Fair	dark amber (`--warning`), 60 %	`Abcdef12` (browser test output)
4	Good	green (`--success`), 80 %	–
5	Strong	green (`--success`), 100 %	`Campus#Portal2026` (Figure 3)

The meter is a guide, independent of the pass/fail rules (the code comment says so). For example, a password of 12 or more characters with upper- and lower-case letters and a digit but no special character reaches “Good”, yet the browser still refuses it with “The password needs a special character.” – the rules in Section 5.2 decide.

6. Database and Relationships

The Practical 5 script itself does not access the database and stores nothing in the browser. When a valid form reaches the server, `register_student_account()` (already part of the portal since the Practical 9 integration) writes to two tables of `studenthub_db`:

Table	Columns written	Keys and constraints (database/studenthub_demo.sql)
`users`	`username`, `email`, `password` (bcrypt hash from `password_hash()`), `role` = `'student'`	UNIQUE `username`, UNIQUE `email`, CHECK `chk_users_role` (admin, editor, student)
`students`	`user_id`, `enrollment_no`, `name`, `email`, `course`, `gender`, `semester`, `status` = `'Active'`	UNIQUE `email`, UNIQUE `uq_students_user_id`, UNIQUE `uq_students_enrollment_no`, FK `fk_students_user` → `users.id` ON DELETE SET NULL ON UPDATE CASCADE, CHECK `chk_students_semester` (1–8)

The relationship is users 1 : 0..1 students through `students.user_id` (see `database/README.md`). Before inserting, prepared statements check whether the username or e-mail address is already used. The enrollment number follows the existing scheme – two digits of the admission year, the course code (IT, CS, DS or WD) and three digits – and the next free number is read with `SELECT … FOR UPDATE` inside the transaction, so two simultaneous registrations cannot receive the same number. Either both rows are saved or neither:

studenthub/includes/registration.php (lines 194–211) – one transaction for the account and the student record

```
194          $db->begin_transaction();
195          try {
196              $enrollment = next_enrollment_number($db, $prefix);
```

```
197
198             $stmt = $db->prepare("INSERT INTO users (username, email, password, role) VALUES (?, ?,
    ?, 'student')");
199             $stmt->bind_param('sss', $clean['username'], $clean['email'], $hash);
200             $stmt->execute();
201             $user_id = (int) $stmt->insert_id;
202             $stmt->close();
203
204             $stmt = $db->prepare("INSERT INTO students (user_id, enrollment_no, name, email, course,
    gender, semester, status)
205                                   VALUES (?, ?, ?, ?, ?, ?, ?, 'Active')");
206             $stmt->bind_param('issssi', $user_id, $enrollment, $clean['name'], $clean['email'],
    $clean['course'], $clean['gender'], $clean['semester']);
207             $stmt->execute();
208             $student_id = (int) $stmt->insert_id;
209             $stmt->close();
210
211             $db->commit();
```

If a UNIQUE key still rejects a row (MySQL error 1062, e.g. the same username registered at the same moment), the transaction is rolled back and the user receives a field message instead of an error page. No database change was needed for Practical 5.

7. Validation, Security and Accessibility

7.1 The server remains the authority

Browser checks can be switched off or bypassed, so they only give earlier, friendlier feedback. `validate_registration()` (lines 53–126 of `includes/registration.php`) checks every field again with the rules of the table in Section 5.2, rejects values that are not plain UTF-8 strings (for example `name[]=x`) with "This field contains characters that are not allowed.", and returns the cleaned values together with the errors by field. Its password, confirm-password and terms checks:

studenthub/includes/registration.php (lines 103–115) – server-side rules with the same messages

```
103     if (!isset($errors['password'])) {
104         $problems = password_problems($raw['password']);
105         if ($raw['password'] === '') {
106             $errors['password'] = 'Please choose a password.';
107         } elseif ($problems) {
108             $errors['password'] = 'The password needs ' . implode(', ', $problems) . '.';
109         }
110     }
111     if (!isset($errors['confirm_password']) && !isset($errors['password']) &&
    !hash_equals($raw['password'], $raw['confirm_password'])) {
112         $errors['confirm_password'] = 'The passwords do not match.';
113     }
114     if (($input['agree'] ?? null) !== '1') {
115         $errors['agree'] = 'Please accept the Terms & Conditions.';
```

This was tested directly: `tests/test_forms.py` sends a registration with the weak password `weakpass` straight to the server, without the browser. The server answers HTTP 422 with the policy message and no account is created.

7.2 Security measures on the registration page

- **CSRF:** the form contains the session token (`csrf_field()`); a missing or wrong token gives the message "The registration form expired. Please check your details and submit again." and nothing is saved.
- **Output escaping:** values shown again after a refusal are printed with `e()` (`htmlspecialchars` with `ENT_QUOTES | ENT_SUBSTITUTE`, UTF-8); password fields are never refilled. The script writes messages with `textContent`; its only `innerHTML` is the fixed markup of the meter, which contains no user data.
- **Role:** the SQL statement inserts the literal role `'student'`; the role is never taken from the form.

- **Passwords:** stored only as `password_hash()` hashes; when a registration fails on the server, the error-log entry contains no password or hash (`registration.php`, lines 223–224).
- **Session:** `login_user()` creates a new session ID and a new CSRF token after a successful registration.

7.3 Accessibility

- Visible labels connected with `for`/`id`; required fields are marked with an asterisk, explained by the note “Fields marked * are required.” above the form.
- Messages linked with `aria-describedby`; `aria-invalid="true"` on invalid fields; the error is given as text, not only by colour.
- The strength label is an `aria-live="polite"` region; the server's general error box has `role="alert"`.
- Only native controls are used; after a blocked submit the focus moves to the first invalid field.

8. Output Screenshots

All screenshots are genuine full-page captures of `studenthub/register.php` in Microsoft Edge (headless, driven by Playwright, viewport 1280 px wide), made by `tests/browser_forms.py` on 31 October 2026 against the PHP built-in server and a disposable database. The names and addresses typed are fictional test values.

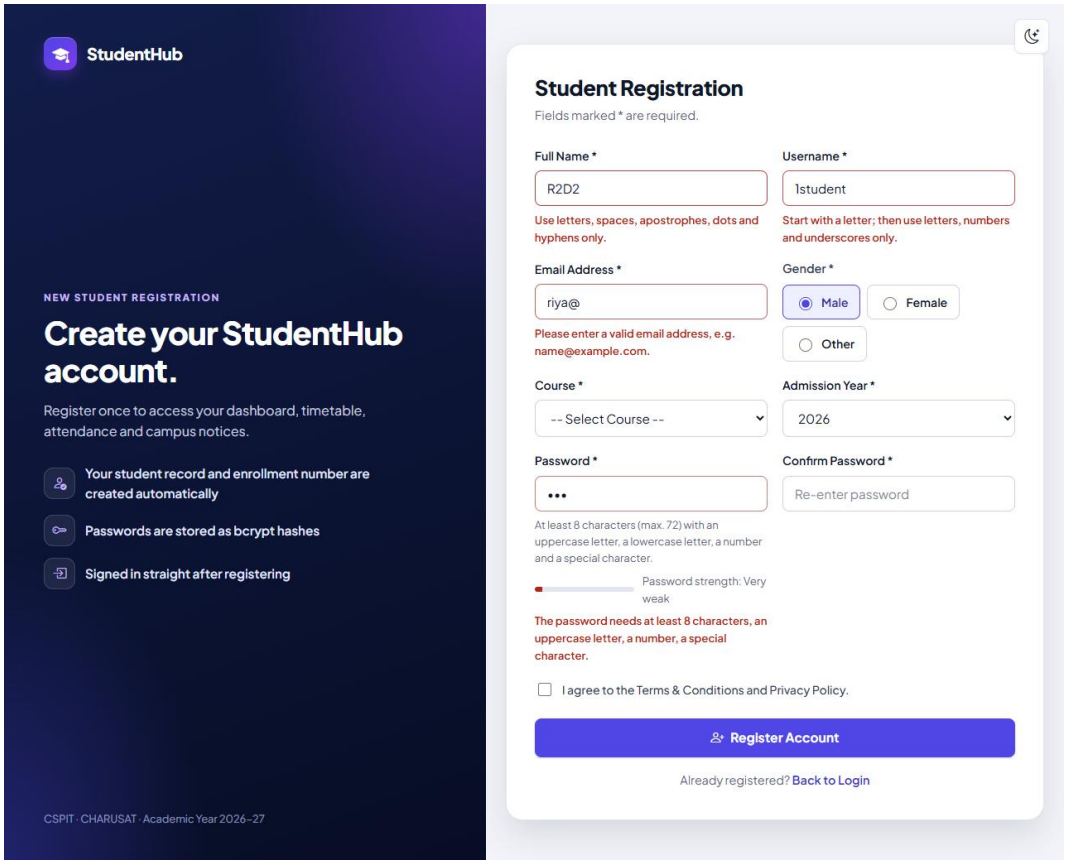


Figure 1: Real-time validation: after typing R2D2, 1student, riya@ and abc and leaving each field, the server-worded messages appear under Full Name, Username, Email Address and Password, and the meter reads “Password strength: Very weak”; Course and the terms checkbox were not touched yet and show no message. No request was sent. (01-register-live-validation.png)

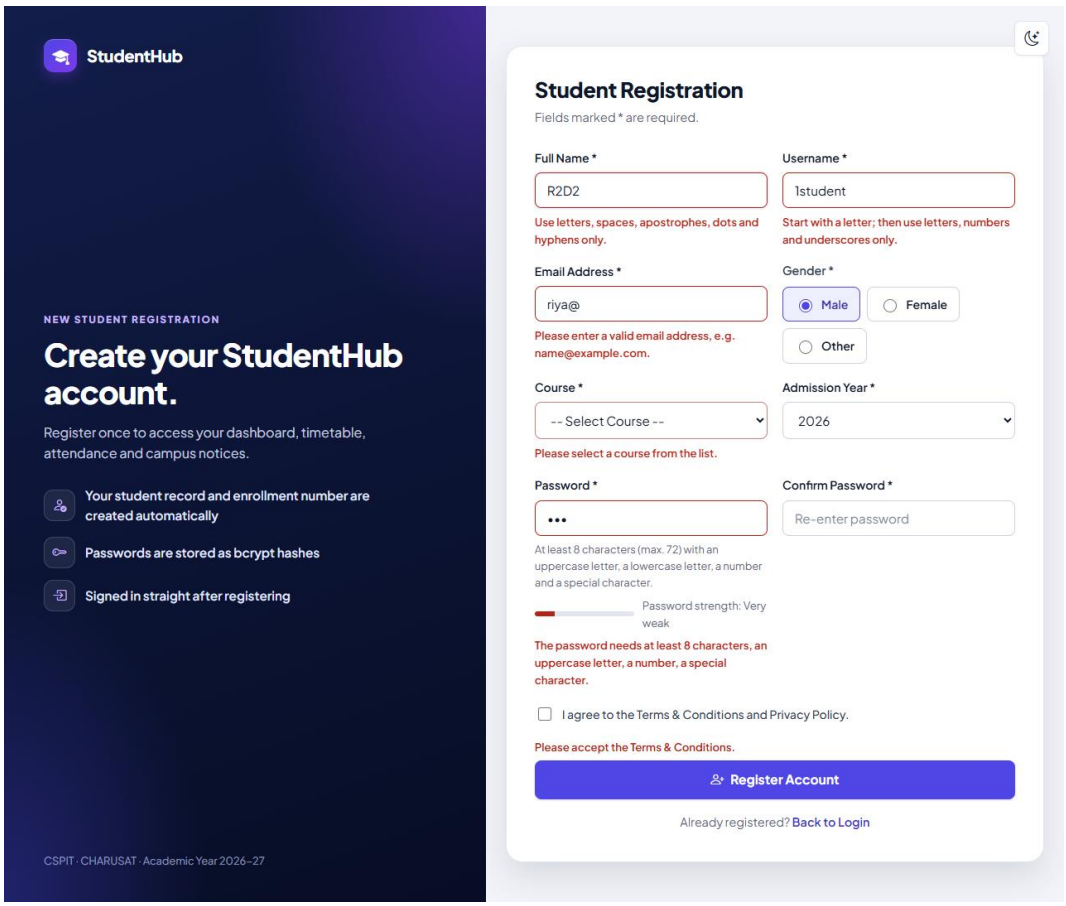


Figure 2: Register Account clicked while rules are broken: the submission is stopped in the browser, the messages "Please select a course from the list." and "Please accept the Terms & Conditions." are added, and the test confirmed that the focus moved to Full Name. Confirm Password shows no message because it is checked only once the password itself is valid. (02-registration-blocked.png)

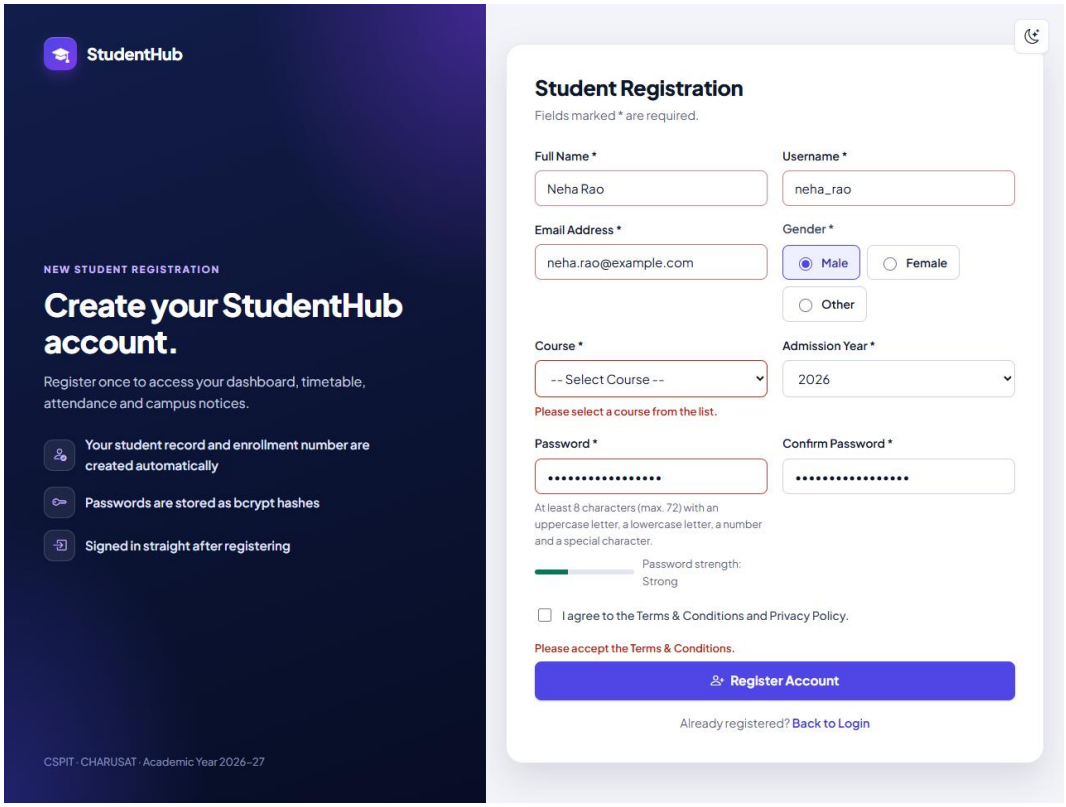


Figure 3: Valid values typed (Neha Rao, neha_rao, neha.rao@example.com, Campus#Portal2026 in both password fields): the messages under name, username, e-mail and password have disappeared and the label reads "Password strength:

Strong". Course and terms are still missing, so their messages remain. In this capture the green bar is drawn only partly, although the script sets 100 % for the top score – auth.css animates the width with a 0.2 s transition. (03-password-strength.png)

9. Testing and Results

Environment of the recorded runs (31 October 2026): Windows 11, PHP 8.2.12 (XAMPP) built-in server on 127.0.0.1, MariaDB 10.4.32, Microsoft Edge in headless mode through Playwright. The suites create a disposable database and remove it at the end; `tests/test_forms.py` also confirms that the live `studenthub_db` was not touched. The complete run of all suites (`tests/run_all_output.txt`) passed 841 checks with 0 failures.

Suite	Scope	Result	Output file
`tests/test_forms.py`	HTTP tests of Practicals 5, 6 and 7 in the portal; 4 checks concern Practical 5	**41 / 41 passed**	`tests/test_forms_output.txt`
`tests/browser_forms.py`	Microsoft Edge checks and screenshots of Practicals 5–7; 4 checks concern Practical 5, two more cover the whole run (no JavaScript errors, no PHP warnings)	**15 / 15 passed**	`tests/browser_forms_output.txt`
`tests/test_auth.py` (regression)	Authentication and registration over HTTP	**132 / 132 passed**	`tests/test_auth_output.txt`
`tests/browser_auth.py` (regression)	Edge test of login and registration; its weak-password check now expects the browser to stop the form (the server refusal is tested over HTTP)	**14 / 14 passed**	`tests/browser_auth_output.txt`

Practical 5 checks, copied from the saved outputs:

- register.php loads the validation script (and keeps server-side validation: novalidate form) – PASS
- the script checks every field with regular expressions / rules matching the server (name, username, email, password policy, confirm, gender, course, year, terms) – PASS
- password strength meter, real-time checks (input/blur) and submit blocking are implemented – PASS
- bypassing the browser checks does not help: the server refuses the weak password (422, nothing stored) – PASS
- Edge: real-time validation: messages appear under name, username, e-mail and password while typing (nothing sent) – PASS
- Edge: password strength meter shows 'Very weak' for 'abc' – PASS
- Edge: submitting with broken rules is blocked in the browser: every rule is listed, focus on the first field, no request sent – PASS
- Edge: the meter grows with the password (Abcdef12 -> Fair, Campus#Portal2026 -> Strong) and the errors disappear – PASS
- Edge (whole run): no JavaScript errors; no PHP warnings or errors in the server log – PASS

Note on the method: the second and third Practical 5 checks of `test_forms.py` search the source of the script for the expected rules and listeners; the behaviour itself is checked in Microsoft Edge. Registration as a whole (account and student record created together, role always `student`, bcrypt hash, enrollment number, audit entry) is covered by the "Registration (Practical 9 integrated)" section of `tests/test_auth_output.txt`.

9.1 Not tested

- Screen readers and a person using a physical keyboard; the ARIA attributes were checked only in the code.
- Browsers other than Microsoft Edge, real phones and narrow (mobile) widths – `browser_forms.py` used a 1280 px wide viewport.
- The colour contrast of the new strength-meter colours was not measured, and the page was not submitted to the W3C validator.
- A complete successful registration in the browser was not part of `browser_forms.py`; it is covered over HTTP by `tests/test_auth.py`.

10. Differences from the Original Standalone Practical and Limitations

Aspect	Standalone Practical-5 (original report)	Integrated portal
Page	`Practical-5/register.html`, static; nothing sent or saved	`studenthub/register.php`; creates the account and the student record
Fields	name, email, mobile number, password, confirm, course, year, gender, terms + CAPTCHA	name, username, e-mail, gender, course, admission year, password, confirm, terms – no mobile number
Password policy	8–64 characters; 3-level meter with a checklist of the rules	8–72 characters (bcrypt limit); 5-level meter; the policy text is shown under the field
Error overview	summary “Please correct N fields” at the top	browser: focus on the first invalid field; server: “Please correct the highlighted fields.”
CAPTCHA	Canvas CAPTCHA (client-side demonstration)	not integrated (Section 10.1)
Server validation	none	every field, CSRF token, duplicate checks, transaction
Tests	24 static, 19 contrast and 103 Edge checks	4 HTTP and 4 Edge checks for Practical 5 within the 41 / 41 and 15 / 15 suites, plus the regression suites

10.1 Canvas CAPTCHA – not integrated

The audit in `TEST_REPORT.md` (section 12) records the decision: “the CAPTCHA was not integrated (a client-side canvas CAPTCHA gives no real protection on a live form; the standalone Practical-5/ keeps it)”. The original report already stated that the code was generated and checked in the browser, so any script could read it, and that it was not accessible to blind users. The portal registration form therefore has no CAPTCHA and no other bot protection; the honeypot field of the portal exists only on the contact form (Practical 7).

10.2 Other limitations

- **Mobile number:** the official problem definition lists a mobile number, but the portal's form has no such field – the `users` and `students` tables have no column to store it.
- **JavaScript switched off:** there is no live feedback, but the server still validates every field and shows its messages.
- **Meter vs. rules:** the meter is only a guide; the password policy alone decides whether a password is accepted.
- **E-mail:** the browser's regular expression and the server's `filter_var()` are different checks; the server's result counts.

11. Conclusion

The StudentHub registration page `studenthub/register.php` now validates all its fields in real time with JavaScript regular expressions and rules that match the server: messages appear directly below the fields with the server's wording, a five-level strength meter follows the password as it is typed, and the form is not sent while a rule is broken, with the focus moved to the first problem. The PHP server still checks every value, so a request that bypasses the browser is refused (HTTP 422, nothing stored), and a valid registration creates the account and its student record in one transaction.

The intermediate extension (real-time validation) is integrated; the advanced extension (Canvas CAPTCHA) was deliberately left in the standalone folder because it gives no real protection on a live form. The integration passed the HTTP suite (41 / 41), the Edge suite (15 / 15) and the authentication regression suites (132 / 132 and 14 / 14), and the screenshots show the invalid, blocked and corrected states of the real form.

12. References

- ITUE203 Web Development Frameworks – Practical List 2026-27, Practical 5: “Registration Form with Frontend Validation and User-Friendly Error Handling”, CSPIT, CHARUSAT.
- StudentHub repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 – folders `studenthub/` (portal), `Practical-5/` (original standalone version), `tests/`, `evidence/practical-05-07-integration/`; `TEST_REPORT.md` section 12.
- MDN – Client-side form validation: https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation
- MDN – Regular expressions: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions
- MDN – Unicode character class escape (\p{…}): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Unicode_character_class_escape
- W3C WAI – Forms Tutorial: https://www.w3.org/WAI/tutorials/forms/
- PHP Manual – password_hash: https://www.php.net/manual/en/function.password-hash.php
- PHP Manual – filter_var and validate filters: https://www.php.net/manual/en/filter.filters.validate.php